Food Bridge: An Agentic AI Platform for Real-Time Surplus Food Matching

Donation Intake and Eligibility

Individual Design Specification

Version 0.3 • October 2026 • CMPE 295A • Advisor: Prof. Chandrasekhar Vuppalapati

Mani Mokshith Noonety

ID - 019100458

Email - manimokshithn.oonety@sjsu.edu

Contents

- Version History
- Introduction
  - Individual Contribution and Assignment Compliance
  - Scope and Ownership
- References
- Requirements
  - Functional Requirements
  - Nonfunctional Requirements
- Functional Overview
  - Decision Outcomes
- Configuration and External Interfaces
  - Configuration
  - API Interface
  - External and Internal Dependencies
- Debug
  - Logging
  - Counters
- Implementation
  - Proposed Component Structure
  - Data Structures
  - Urgency Scoring
  - Candidate-Recipient Eligibility
  - Eligibility Rule Evaluation
  - State Transitions
  - Implementation Subtasks
- Testing
  - General Approach
  - Unit Tests
  - Functional and Integration Tests
- Appendix
  - Open Design Decisions

Version History

Version	Date	Author	Changes
0.1	October 10, 2026	Mani Mokshith Noonety	Initial design specification defining the Donation Intake and Eligibility subsystem and its role in the FoodBridge workflow.
0.2	October 21, 2026	Mani Mokshith Noonety	Expanded the design with detailed components, API interfaces, data structures, eligibility rules, state transitions, and implementation subtasks.
0.3	October 28, 2026	Mani Mokshith Noonety	Refined the subsystem for efficiency and reliability by adding urgency thresholds, idempotency, retry behavior, observability metrics, failure handling, and broader test coverage.

Introduction

FoodBridge AI is an agentic platform that helps move surplus food from donors to recipient organizations before the food becomes unusable. This design specification defines the Donation Intake and Eligibility subsystem, which is the first controlled stage of the donation-to-pickup workflow.

The subsystem converts a donor's free-text listing into a structured donation record, validates the required information, estimates urgency, and applies deterministic eligibility checks before the listing is sent to matching and routing. The design intentionally separates extraction assistance from safety and consistency rules. A model may suggest structured values, but it cannot override a failed expiration, timing, capacity, dietary, or food-safety check.

This document is an individual contribution. It adds detailed requirements, schemas, API behavior, validation logic, error handling, observability, implementation tasks, and tests beyond the high-level design in the project workbook.

Individual Contribution and Assignment Compliance

This section makes the individual ownership, additional design detail, and non-overlap boundaries explicit for the Development submission.

- Individual ownership: I own the detailed design of the Donation Intake and Eligibility subsystem, including donor intake, extraction contracts, normalization, urgency scoring, deterministic validation, candidate-recipient filtering, persistence, handoff, observability, and tests.

- Individual evidence: The requirements, API contracts, data structures, eligibility rules, state transitions, failure behavior, metrics, implementation subtasks, and unit/integration tests in this document are my individual design contribution.

- Dependencies: The subsystem depends on the API Gateway/JWT service, Orchestrator and queue, extraction provider, PostgreSQL/PostGIS, time service, and recipient profiles. It returns validated results to the Orchestrator and does not own matching/ranking, routing, reservation, driver assignment, or pickup completion.

- Workbook boundary: Chapter 6 defines the project-level architecture and agent responsibilities. This document adds field-level requirements, interfaces, rule behavior, schemas, failure handling, and test definitions rather than repeating only the workbook diagrams.

- Non-overlap boundary: This specification stops before semantic ranking, geospatial routing, optimistic reservation locking, notifications, and pickup execution. Those areas remain outside this individual contribution and can be covered by other team members.

Scope and Ownership

Included in this specification	Outside this specification
Donor listing intake, extraction, syntactic normalization, urgency scoring, deterministic donation validation, candidate-recipient eligibility filtering, audit persistence, and handoff to the Orchestrator.	Semantic embedding normalization, recipient ranking, geospatial routing, reservation locking, driver assignment, notifications, and pickup completion.
Interfaces and result contracts used to return eligible candidate recipients and the validated donation to the Orchestrator.	Implementation of the other agents and their internal ranking, routing, reservation, or pickup logic.

References

The following project materials define the surrounding architecture and assignment format:

- FoodBridge Workbook 2, especially Chapter 5 Project Architecture and Chapter 6 Project Design.

- FoodBridge Workbook 2, Table 2.1, which defines the Intake and Eligibility Agent responsibility.
- Design Specification template supplied for the individual Development submission.
- Project requirements for free-text donation creation, urgency scoring, deterministic eligibility, and status tracking.

Requirements

The requirements below define the expected behavior of the Donation Intake and Eligibility subsystem. Requirement identifiers are used in the implementation and testing sections.

Functional Requirements

ID	Requirement	Priority
DI-01	The system shall accept a donor listing containing free-text food description, quantity, pickup location, and pickup window.	Must
DI-02	The system shall validate authentication, required fields, field lengths, supported units, and location format before processing.	Must
DI-03	The extraction component shall convert free text into structured food type, quantity, packaging, expiry or safe-use time, allergens, and notes.	Must
DI-04	The system shall preserve the original donor text together with extracted values for auditability and correction.	Must
DI-05	The system shall calculate an urgency category and numeric urgency score using time remaining, pickup window, and configured thresholds.	Must
DI-06	The deterministic rules engine shall reject or flag listings that are expired, missing critical safety information, outside the pickup window, or otherwise invalid.	Must
DI-07	The system shall store eligible donations with a stable donation identifier and status of ELIGIBLE.	Must
DI-08	The system shall send an eligible donation event to the Orchestrator without placing a model-generated match directly into the database.	Must
DI-09	The system shall return actionable validation errors without exposing internal model prompts or sensitive system details.	Must
DI-10	The system shall make extraction and eligibility decisions idempotent so that a retried request does not create duplicate donations.	Must
DI-11	The system shall filter candidate recipients using capacity, operating hours, dietary compatibility, food category, and pickup feasibility before ranking.	Must

ID	Requirement	Priority
DI-12	The Intake and Eligibility Agent shall return the validated donation and eligible candidate-recipient set to the Orchestrator; it shall not rank or reserve a recipient.	Must

Nonfunctional Requirements

ID	Requirement	Target or constraint
NFR-01	Safety boundary	Deterministic rules must run after extraction and before matching.
NFR-02	Traceability	Each decision must be traceable to a donation ID, request ID, rule result, and model version when applicable.
NFR-03	Latency	Normal validation should return within 2 seconds excluding a temporary external model outage.
NFR-04	Reliability	A retried request with the same idempotency key must produce one logical donation.
NFR-05	Privacy	Donor contact information must not be written to application logs in plain text.
NFR-06	Configurability	Urgency thresholds, accepted units, and required fields must be configurable without code changes.

Functional Overview

The subsystem has five logical stages: submission, extraction, normalization, deterministic validation, and handoff. The stages are designed so that a low-confidence model output creates a reviewable validation result rather than silently becoming an unsafe donation record.

Stage	Input	Processing	Output
1. Submission	Donor text and form metadata	Authenticate request, validate request shape, assign request ID.	Accepted intake request or client validation error.
2. Extraction	Free-text description	Extract food type, quantity, expiry/safe-use time, allergens, packaging, and confidence values.	Structured candidate fields with confidence values.
3. Normalization	Candidate fields	Normalize units, timestamps, location, and controlled labels; semantic embeddings remain in Matching and Routing.	Canonical donation fields.
4. Eligibility	Canonical fields	Apply donation-safety and candidate-recipient rules for expiry, timing, capacity, dietary fit, operating hours, quantity, and completeness.	Eligible, rejected, or needs-review decision with rule results.
5. Persistence and handoff	Eligible donation	Persist the result and return an idempotent intake result to the Orchestrator for the next workflow step.	Donation ID, decision, eligible candidate set, and audit record.

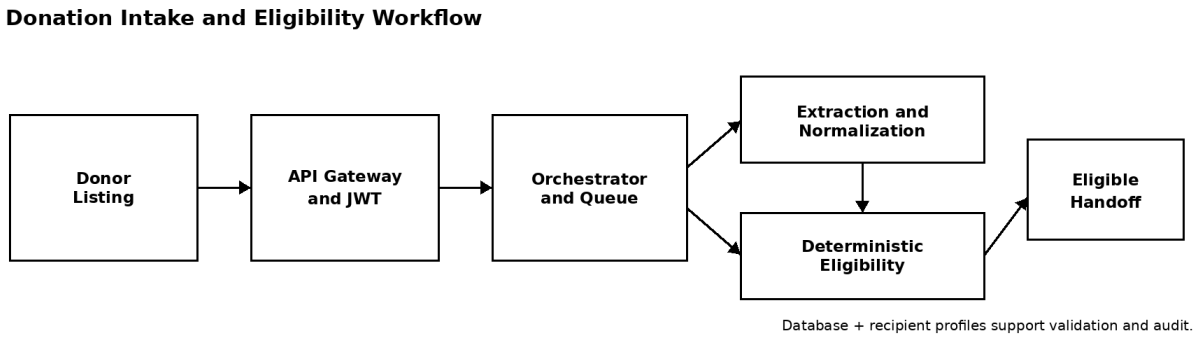


Figure 1. Donation Intake and Eligibility workflow

Decision Outcomes

Outcome	Meaning	Next action
ELIGIBLE	All mandatory rules passed.	Persist and send to matching workflow.
REJECTED	The listing cannot safely or correctly enter matching.	Return reason codes and preserve an audit record.
NEEDS_REVIEW	The listing is potentially valid but extraction confidence or required information is insufficient.	Ask the donor for clarification or route to an operator review queue.

Configuration and External Interfaces

Configuration

Configuration	Example	Purpose
Urgency thresholds	critical: <= 2 hours; high: <= 6 hours	Convert time remaining into a consistent urgency category.
Accepted quantity units	meal, serving, box, pound, kilogram	Prevent inconsistent quantity comparisons.
Minimum confidence	0.85 for expiry and quantity	Send low-confidence extraction to NEEDS_REVIEW.
Maximum text length	2,000 characters	Limit abuse and model input size.
Required safety fields	food type, quantity, pickup location, pickup window	Prevent incomplete listings from entering matching.

API Interface

The API Gateway authenticates the donor and submits the request to the Orchestrator. The Orchestrator dispatches the Intake and Eligibility task through the queue; the agent returns its validated result and eligible candidate-recipient set to the Orchestrator.

```
POST /api/v1/donations
Headers: Authorization: Bearer <JWT>
         Idempotency-Key: 8d7b...

Request:
{
  "description": "20 boxed vegetarian meals, pickup by 8 PM",
  "pickup_window": {"start": "2026-10-27T17:00:00Z", "end": "2026-10-27T20:00:00Z"},
  "location": {"latitude": 37.3352, "longitude": -121.8811},
```

```
  "notes": "Contains dairy"
}
```

Response 202 Accepted:

```
{
  "donation_id": "don_01J...",
  "workflow_id": "wf_01J...",
  "status": "PROCESSING",
  "next_step": "INTAKE_ELIGIBILITY_QUEUED"
}
```

GET /api/v1/donations/{donation_id} returns the final decision, urgency, eligible candidate-recipient set, and reason codes after processing.

External and Internal Dependencies

Dependency	Use	Failure behavior
LLM or extraction model	Suggest structured values from donor text.	Retry once, then return NEEDS_REVIEW; never bypass deterministic rules.
PostgreSQL/PostGIS	Persist donation data and canonical location.	Do not acknowledge success until the transaction is committed.
Redis and queue	Dispatch eligible donation event.	Keep donation ELIGIBLE and retry publication using an outbox or retry worker.
API Gateway and JWT service	Authenticate donor and enforce request boundaries.	Return 401 or 403 without invoking extraction.
Time service	Normalize timestamps and calculate time remaining.	Reject ambiguous timestamps rather than guessing.

Debug

Logging

Logs must support diagnosis without exposing donor contact information or raw sensitive notes. Every log entry should include request_id, donation_id when available, component, operation, outcome, and duration_ms. Raw free text should be omitted or redacted from normal logs.

Event	Required fields	Example outcome
intake.received	request_id, donor_id, payload_size	Request accepted for extraction.
extraction.completed	request_id, model_version, confidence_summary, duration_ms	Quantity and expiry extracted with high confidence.
validation.completed	donation_id, rule_results, decision	Rejected because pickup window is expired.
donation.persisted	donation_id, status, transaction_id	Donation stored as ELIGIBLE.
event.publish_failed	donation_id, event_id, retry_count, error_type	Queue unavailable; retry scheduled.

Counters

Metric	Description	Why it matters
intake_requests_total	Total listing requests by result code.	Measures usage and client error rate.
extraction_confidence	Distribution of confidence by field.	Detects prompt, model, or input-quality degradation.

Metric	Description	Why it matters
eligibility_decisions_total	Eligible, rejected, and needs-review counts.	Shows whether rules are too strict or too permissive.
validation_latency_ms	Time spent in validation stages.	Supports the latency requirement.
duplicate_idempotency_total	Requests safely recognized as duplicates.	Confirms retry safety.
eligible_event_retry_total	Publication retries for eligible donations.	Detects queue or downstream instability.

Implementation

Proposed Component Structure

Component	Responsibility	Interface
DonationController	Receives authenticated requests and validates request shape.	HTTP POST /api/v1/donations
ExtractionService	Converts free text into candidate structured fields.	extract(description)
NormalizationService	Canonicalizes units, timestamps, categories, and location.	normalize(candidate)
EligibilityRules	Applies deterministic rules and returns rule-level results.	evaluate(canonical_donation)
DonationRepository	Persists donation, audit record, and status transition.	create(), find_by_idempotency_key()
EligibilityEventPublisher	Publishes the eligible event exactly once logically.	publish(donation.eligible)

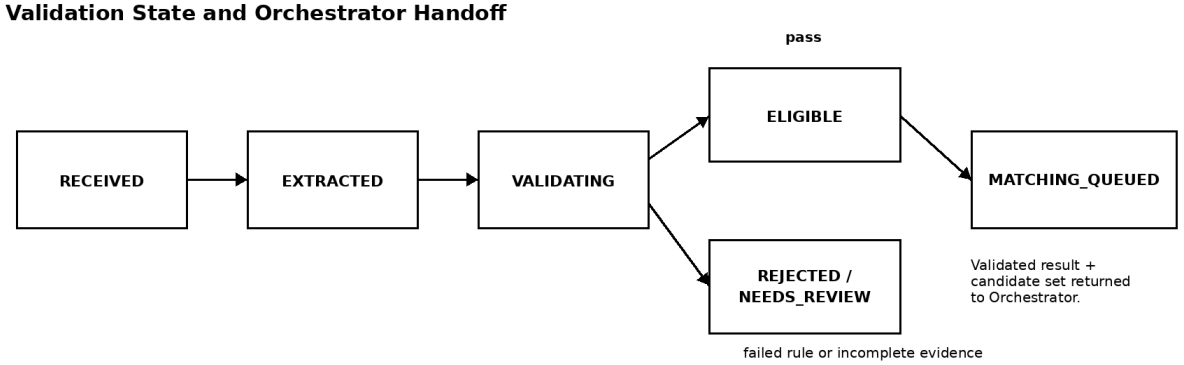


Figure 2. Validation state and Orchestrator handoff

Data Structures

The following logical model adds field-level definitions to the workbook entity model. The exact database type may be implemented using PostgreSQL JSONB for extraction metadata and typed columns for queryable fields.

Donation
- donation_id: UUID, primary key
- donor_id: UUID, foreign key
- original_description: text
- food_category: enum
- quantity_value: decimal
- quantity_unit: enum

- packaging_type: enum, nullable
- expiry_time: timestamp with timezone, nullable
- pickup_window_start: timestamp with timezone
- pickup_window_end: timestamp with timezone
- urgency_score: decimal(4,3)
- urgency_category: enum(CRITICAL, HIGH, NORMAL)
- allergen_tags: array<string>
- location: geography(Point, 4326)
- status: enum(RECEIVED, PROCESSING, ELIGIBLE, REJECTED, NEEDS_REVIEW, MATCHING_QUEUED)
- extraction_model_version: string, nullable
- idempotency_key: string, unique
- created_at: timestamp with timezone
- updated_at: timestamp with timezone

Urgency Scoring

The urgency score is calculated from normalized time remaining until the safe-use or expiry deadline and the pickup-window end. The configured thresholds produce CRITICAL (0–2 hours), HIGH (>2–6 hours), and NORMAL (>6 hours). If the deadline is missing, contradictory, or already passed, the rules engine does not assign a reassuring urgency value; it returns REJECTED or NEEDS_REVIEW according to the food-safety configuration.

Input	Processing rule	Output
Safe-use/expiry time	Use the earliest applicable safety deadline.	Safety deadline
Pickup-window end	Compare the deadline with the time available for pickup.	Feasibility constraint
Threshold configuration	Apply configured critical/high/normal boundaries.	Urgency category and score

Candidate-Recipient Eligibility

After donation-level safety validation, the agent filters recipient profiles before the Matching and Routing Agent ranks them. The filter is deterministic and produces reason codes for every excluded recipient.

Rule ID	Recipient condition	Pass condition	Failure result
CR-01	Capacity	Available capacity is at least the donation quantity.	Exclude recipient: CAPACITY_INSUFFICIENT
CR-02	Operating hours	Recipient is open during the feasible pickup interval.	Exclude recipient: OUTSIDE_OPERATING_HOURS
CR-03	Dietary compatibility	Donation allergens and dietary attributes satisfy recipient constraints.	Exclude recipient: DIETARY_MISMATCH
CR-04	Pickup feasibility	Recipient location and time window can be passed to routing.	Exclude recipient: PICKUP_NOT_FEASIBLE

Eligibility Rule Evaluation

Rule ID	Rule	Pass condition	Failure decision
ER-01	Required fields	Description, quantity, location, and pickup window exist.	REJECTED

Rule ID	Rule	Pass condition	Failure decision
ER-02	Quantity validity	Quantity is positive and uses a supported unit.	REJECTED
ER-03	Time validity	Pickup end is after pickup start and current time is before pickup end.	REJECTED
ER-04	Food safety	Food category is supported and required expiry information is present.	REJECTED or NEEDS_REVIEW
ER-05	Extraction confidence	Critical fields meet the configured confidence threshold.	NEEDS_REVIEW
ER-06	Location validity	Latitude and longitude are within valid geographic ranges.	REJECTED
ER-07	Idempotency	The same key maps to the same logical request.	Return existing result

State Transitions

Current state	Event	Next state
RECEIVED	Extraction starts	PROCESSING
PROCESSING	All rules pass	ELIGIBLE
PROCESSING	Critical rule fails	REJECTED
PROCESSING	Low confidence or missing clarification	NEEDS_REVIEW
NEEDS_REVIEW	Donor supplies correction	PROCESSING
ELIGIBLE	Orchestrator accepts event	MATCHING_QUEUED

Implementation Subtasks

1. Define request, response, and event schemas with versioned contracts.
2. Implement request validation and idempotency-key handling.
3. Implement extraction adapter with a mocked provider for local tests.
4. Implement normalization for quantity units, timestamps, allergens, and food categories.
5. Implement deterministic eligibility rules with rule-level reason codes.
6. Implement PostgreSQL persistence and status-history records.
7. Implement eligible-event publication and retry behavior.
8. Add metrics, structured logs, and redaction checks.
9. Add unit, integration, and failure tests described below.

Testing

General Approach

Testing will separate model-assisted extraction from deterministic eligibility. Extraction tests will use fixed provider responses and representative donor descriptions so that changes in the model adapter do not make the core rules nondeterministic. Eligibility tests will directly exercise canonical structured inputs and must pass even when the model is unavailable.

The primary integration test will submit a donation through the API, persist the result, inspect the status history, and verify that exactly one eligible event is published. Boundary tests will use timestamps immediately before, at, and after expiry. Failure-injection tests will simulate model timeout, database failure, duplicate requests, and queue publication failure.

Unit Tests

Test ID	Scenario	Expected result
UT-01	Valid description: "20 boxed vegetarian meals, pickup by 8 PM".	Structured fields are extracted and the request proceeds to validation.
UT-02	Missing quantity.	Decision is REJECTED with MISSING_QUANTITY.
UT-03	Quantity is zero or negative.	Decision is REJECTED with INVALID_QUANTITY.
UT-04	Unsupported unit such as "a lot".	Decision is NEEDS_REVIEW or REJECTED according to configuration.
UT-05	Pickup end is before pickup start.	Decision is REJECTED with INVALID_PICKUP_WINDOW.
UT-06	Pickup window has already ended.	Decision is REJECTED with EXPIRED_PICKUP_WINDOW.
UT-07	Expiry time is in the past.	Decision is REJECTED with EXPIRED_FOOD.
UT-08	Expiry is missing for a food category that requires it.	Decision is NEEDS_REVIEW or REJECTED according to safety configuration.
UT-09	Latitude is 95 degrees.	Decision is REJECTED with INVALID_LOCATION.
UT-10	Extraction confidence for quantity is below threshold.	Decision is NEEDS_REVIEW.
UT-11	Same idempotency key is submitted twice.	One donation is created and the existing result is returned.
UT-12	Model provider times out.	The request is retried and then moves to NEEDS_REVIEW without bypassing rules.
UT-13	Recipient capacity is below the donation quantity.	Recipient is excluded with CAPACITY_INSUFFICIENT.
UT-14	Recipient is closed during the pickup window.	Recipient is excluded with OUTSIDE_OPERATING_HOURS.
UT-15	Donation conflicts with recipient dietary constraints.	Recipient is excluded with DIETARY_MISMATCH.
UT-16	Donation has two hours remaining before the applicable deadline.	Urgency is CRITICAL and the configured score is recorded.

Functional and Integration Tests

Test ID	Test flow	Pass condition
FT-01	Submit valid donor listing through the API.	One ELIGIBLE donation is persisted and one event is published.
FT-02	Submit listing with invalid safety information.	No matching event is published and the donor receives a reason code.
FT-03	Crash after database commit but before event acknowledgement.	Retry publishes the existing event without creating a duplicate donation.
FT-04	Queue is unavailable for three publication attempts.	Donation remains traceable and an event retry is scheduled.
FT-05	Submit two requests concurrently with one idempotency key.	Exactly one logical donation is stored.
FT-06	Submit a listing containing allergen notes.	Allergen tags are preserved and available to downstream matching.
FT-07	Orchestrator dispatches intake and receives the validated donation plus eligible candidate-recipient set.	Intake does not rank or reserve; Matching and Routing receives only candidates that passed deterministic eligibility.

Appendix

Example Eligibility Response

```
{
  "donation_id": "don_01J8F3Q2",
  "status": "REJECTED",
  "reason_codes": ["EXPIRED_PICKUP_WINDOW"],
  "field_errors": {
    "pickup_window.end": "Pickup window must be in the future."
  },
  "trace": {
    "request_id": "req_01J8F3Q1",
    "ruleset_version": "eligibility-0.1"
  }
}
```

Example Eligible Event

```
{
  "event_id": "evt_01J8F3Q3",
  "event_type": "donation.eligible",
  "event_version": "1",
  "donation_id": "don_01J8F3Q2",
  "urgency_category": "HIGH",
  "pickup_window_end": "2026-10-27T20:00:00Z",
  "occurred_at": "2026-10-27T17:02:13Z"
}
```

Open Design Decisions

- The initial implementation should use a configured extraction provider interface so the provider can be replaced without changing the validation rules.
- The team should confirm the final supported food categories and expiry requirements with the project advisor before implementation.
- The team should decide whether NEEDS_REVIEW is handled by an operator dashboard or returned to the donor for clarification in the first MVP.